Data card: PokéVault

This card documents every dataset PokéVault uses or produces. It is structured after Google's Data Cards Playbook and Gebru et al.'s "Datasheets for Datasets". Facts come from the code in this repository and the published catalogue built on 28 October 2026. Anything not yet confirmed is marked **TODO (Ryley)**.

Contents

Dataset	What it is	Where it lives	Personal data
1. Card catalogue	Every English Pokémon card and set	Built daily; published with the web app (`catalog/catalog.json`)	Illustrators' credited names only
2. Card picture fingerprints	A 286-bit fingerprint of each card's picture	Published with the web app (`catalog/prints.bin`)	None
3. Market prices	Card prices from TCGplayer, Cardmarket and optionally PriceCharting, plus exchange rates	Published files and API database tables	None
4. Accounts and collection database	Users, sessions, the shared collection, binders, want list, audit log	Cloudflare D1 (the API's database)	Yes: see section 4
5. Simulated page-photo evaluation set	Generated photos of binder pages with known contents	Generated locally, not committed	None

1. Card catalogue

Summary. A compact index of every English Pokémon Trading Card Game card that the app can search, scan and value. The 28 October 2026 build holds 21,256 cards in 205 sets, 399 illustrators, 31 rarities and 3 categories (Pokémon, Trainer, Energy), in about 1.2 MB of JSON.

Source and provenance.

- Sets, cards, rarities, illustrators, HP and printings come from the TCGdex API (`https://api.tcgdex.net/v2/en`, REST and GraphQL).
- For about 50 sets where TCGdex marks every card "normal" (mostly Black & White, XY and Sun & Moon), printings are inferred from each card's rarity in the pokemon-tcg-data files.
- Card pictures are not copied. The catalogue stores a link base (`https://assets.tcgdex.net/en`). Where TCGdex has no picture, the app falls back to `images.pokemontcg.io` at display time.

Licence and terms.

- Card names, pictures and text are the property of Nintendo, Creatures and GAME FREAK (The Pokémon Company). PokéVault is an unofficial fan project.
- The terms of use for TCGdex's data and of the pokemon-tcg-data files: **TODO (Ryley): confirm and cite.**

Collection and processing. `scripts/build-catalog.mjs` runs daily in GitHub Actions (`.github/workflows/deploy-web.yml`). It:

1. Fetches all sets and cards, skipping the digital-only Pokémon TCG Pocket series.
2. Normalises illustrator names, so spelling variants of the same artist are grouped together.
3. Works out each card's printings. These are the plain printings (regular, holo, reverse holo, 1st Edition, W promo) plus special printings (special foils, stamps and metal cards). The build classifies each printing as from the set's packs or as a promo, and links stamped promo cards to the set card they reprint (by name and attacks).
4. Corrects known errors in the source data. For example, every Fossil card is listed with a "galaxy" foil, and some gold secret rares are listed twice.

Schema. `catalog.json` is an object with these fields:

Field	Type	Meaning
`version`	number	Format version (1)
`builtAt`	ISO 8601 date-time	When the build ran
`source`	string	Attribution text
`imageBase`	URL	Base address for card pictures
`columns`	string[]	Names of the first ten fields of each card row
`sets`	object[]	`id`, `name`, `serie`, `serieName`, `date`, `official` (printed set size), `total`, `code`, `logo`, `symbol`
`artists`, `rarities`, `categories`	string[]	Lookup tables that card rows point into
`imageOverrides`	object	Card id → picture URL, where TCGdex's address doesn't fit the pattern
`cards`	array[]	One row per card (below)
`printings`	number	1 when rows carry promo printings (fields 11–13)

Each card row is an array:

Position	Field	Example (Kingdra, Paradox Rift)
1	Card id	`"sv04-032"`
2	Name	`"Kingdra"`
3	Number as printed	`"032"`
4	Set (index into `sets`)	`182`
5	Illustrator (index into `artists`, −1 if none)	`354`
6	Rarity (index into `rarities`)	`19`
7	Category (index into `categories`)	`1`
8	Has a picture (1 or 0)	`1`
9	HP (0 if none)	`150`
10	Plain printings, as bits (regular, holo, reverse, 1st Edition, W promo)	`6` (holo and reverse)
11	Special printings (or 0)	`["holo~cosmos"]`
12	Printings found only in promos (or 0)	`["holo~cosmos"]`
13	For a stamped promo card, the set card it reprints (or 0)	`0`

`meta.json` records the build time, card, set and artist counts, and a hash of each published file. The app uses it to know when to download a new catalogue.

Personal or sensitive data. None, apart from illustrators' names as credited on the cards. They are public professional attributions and are used only to group and browse cards by artist.

Intended uses. Searching for cards, identifying scanned cards, building binders and set checklists, and showing which printings a card comes in.

Out-of-scope uses. It is not an authoritative record of print runs or rarity, and not a source of card pictures or text for redistribution. It is not suitable for Japanese, Korean or Chinese cards, which it doesn't include.

Known limitations and biases.

- English cards only.
- Accuracy depends on TCGdex. Errors found so far are corrected in the build, but others will exist.
- Printings for about 50 older sets are inferred from rarity, not recorded.
- A card without a picture can be found by search or by its printed text, but not by its picture.
- New sets appear only once TCGdex adds them.

Maintenance.

- Rebuilt every day and on every push to `main`. If a build fails, the previously published catalogue is reused.
- The format version and per-file hashes let the app and the build detect changes.
- Owner: Ryley Griffiths.

2. Card picture fingerprints

Summary. A perceptual fingerprint ("pHash") of every card picture in the catalogue. The scanner compares the card it sees against these to recognise it on the device, with no image sent anywhere. The published file is 765,216 bytes: 36 bytes for each of the 21,256 cards, in catalogue order.

Source and provenance. Computed by `scripts/build-catalog.mjs` from each card's small picture on TCGdex (`low.jpg`). Fingerprints from the previous build are reused, so only new cards' pictures are downloaded.

Licence and terms. Derived from the card pictures (see dataset 1). Only the fingerprints are published, not the pictures. **TODO (Ryley): confirm this use is within TCGdex's terms.**

Collection and processing. Defined in `shared/cardprint.ts`, which both the build and the app use:

1. Convert the picture to greyscale.
2. Take two regions: the whole card, and the artwork area (10–90% across, 11–50% down).
3. Resample each to 48 × 48 pixels and take the 12 × 12 lowest frequencies of its discrete cosine transform, dropping the constant term.
4. Set each bit to whether that coefficient is above the region's median.

This gives 2 × 143 = 286 bits, stored as nine 32-bit words, with all zeros meaning "no picture". Pictures are compared by Hamming distance (the number of differing bits).

Schema. A flat binary file of little-endian unsigned 32-bit words, `9 × number of cards` long. Card i's fingerprint is words `9i` to `9i + 8`. `meta.json` records the fingerprint format version, so a format change forces every fingerprint to be recomputed.

Personal or sensitive data. None.

Intended uses. Recognising a card from a photo, in the browser.

Out-of-scope uses. Recovering card pictures (a fingerprint is far too coarse) and identifying anything other than cards in the catalogue.

Known limitations and biases.

- A fingerprint captures broad light and dark shapes, so reprints with the same artwork have near-identical fingerprints; the app then reads the printed number.
- Cards that are mostly flat colour (some Energy cards, for example) are harder to tell apart.
- Glare and holographic foil change a photo's fingerprint.

See the model card for how recognition uses it.

Maintenance. Rebuilt with the catalogue. Owner: Ryley Griffiths.

3. Market prices

Summary. The prices the app values cards with. It has three parts:

- **Published with the catalogue:**
  - `prices.json`: one near-mint price for each card's standard printing, for set values.
  - `cardmarket/<set>.json`: Cardmarket prices per card and per special printing.
- **Kept by the API in its database:**
  - `card_prices`: the latest price points.
  - `price_history`: one point per card, source, printing and grade a day.
  - `fx_rates`: the European Central Bank's latest rates.
  - `pricecharting_map`: which PriceCharting product a card matches.
  - `card_cache`: card details cached from TCGdex.

Source and provenance.

Source	What	How it is fetched
TCGplayer, via TCGdex	US market prices (USD) for English cards	From TCGdex's card data
Cardmarket	European prices (EUR) for cards in any language	Cardmarket's public product list and daily price guide (`downloads.s3.cardmarket.com/productCatalog`)
PriceCharting (optional)	Graded prices (PSA, BGS, CGC, SGC)	PriceCharting's API, with a paid token kept as a server secret
European Central Bank	Daily euro reference rates	The ECB's daily XML file, fetched by the API at most twice a day
The users	eBay sold prices they log	Entered in the app (stored with the collection, dataset 4)

Licence and terms. Prices belong to their sources. **TODO (Ryley): confirm the terms for Cardmarket's public price files, TCGplayer prices via TCGdex, PriceCharting's API and the ECB's reference rates.**

Collection and processing.

- **Checking Cardmarket links.** TCGdex links some cards to the wrong Cardmarket product. For example, it links EX Deoxys's Gold Star Rayquaza to the plain Rayquaza from the same set. So

the build checks every link against Cardmarket's product name and attacks and corrects the wrong ones. Where several products look alike, it uses the order Cardmarket numbers them in. Each build prints how many links it confirmed, corrected or could not find.

- **Special printings.** A special printing (a cosmos holo, a stamped card) is priced from its own Cardmarket product. Until it has one, the app uses the plain printing's price and says so.
- **Set values.** `prices.json` gives each card's standard printing in USD (TCGplayer) or else EUR (Cardmarket), in catalogue order.
- **Conversion.** The API converts prices into the collection's currency with the ECB's rates.

Schema.

- **`prices.json`:** `{ version, updated, usd: (number | null)[], eur: (number | null)[] }`, one entry per catalogue card. In the 28 October 2026 build, 17,441 cards have a USD price and 3,221 a EUR price.
- **`cardmarket/<set>.json`:** `{ version, updated, cards }`. In `cards`, a card id (or `cardId|printing` for a special printing) maps to either 0 (no product) or `[product id, trend, low, 30-day average, holo trend, holo low, holo 30-day average]` in EUR. Cardmarket's "holo" prices are the reverse holo's.
- **API tables:** defined in `api/migrations/0001_init.sql` and `0002_currency.sql`. Prices are stored with their currency and the time they were fetched.

Personal or sensitive data. None.

Intended uses. Approximate valuation of a personal collection, set completion costs and a want-list budget.

Out-of-scope uses.

- Pricing for sales or insurance.
- Financial or investment decisions.
- A record of what any card sold for: the prices are aggregates from the sources, not transactions.

Known limitations and biases.

- Markets differ. TCGplayer reflects the US market for English cards; Cardmarket reflects European sales of a card in any language.
- Prices for rarely traded cards are noisy, and averages lag sudden moves.
- Set values assume one near-mint copy of each card's standard printing.
- Catalogue prices are refreshed when the site rebuilds (TCGplayer's every 6 days, Cardmarket's daily). Prices for cards in the collection are refreshed by the API (see Maintenance).
- The link check reduces, but cannot remove, the chance of a card being matched to the wrong product. A card it can't settle is left without a Cardmarket price rather than given a wrong one.
- Graded prices need a paid PriceCharting subscription; without one, graded copies are valued only from sales the users log.

Maintenance.

- Catalogue prices are rebuilt daily with the catalogue.
- For cards in the collection, the API's hourly job refreshes any price more than 20 hours old, and opening the app refreshes those more than 6 hours old.
- Owner: Ryley Griffiths.

4. Accounts and collection database

Summary. Everything users create: their accounts, the shared collection, binders, the want list, logged sales and an audit log. It is stored in a Cloudflare D1 (SQLite) database behind the API. The schema is defined by the migrations in `api/migrations/`.

Source and provenance. Entered by the app's users. The live instance is run by Ryley Griffiths for a personal collection.

Licence and terms. Not published. The repository contains no user data, only the schema. The test files use made-up `example.com` accounts.

Collection and processing.

- **Accounts:**
  - An account is created with an email address, a display name and a password.
  - The browser stretches the password with PBKDF2-SHA256 (600,000 iterations) before sending it. The server stores only a salted SHA-256 hash of the result, so the password itself never reaches the server.
  - The first account becomes the admin; later sign-ups are "Pending" until an admin approves them.
- **Collection:**
  - Collection items, binders, wants and logged sales record who created them.
  - Every change is written to the audit log.
- **Scans:** camera frames and photos are processed in the browser and are not stored or sent to the API.

Schema (fields that relate to people). Collection data (cards, printings, conditions, prices) is in the same database.

Table	Personal fields	Purpose
`users`	email, display name, password salt and hash, role, failed sign-ins, lock time, created and last sign-in times	Signing in, roles
`sessions`	hash of the session token, user id, created and expiry times, browser user-agent	Staying signed in (30 days)
`rate_limits`	a key containing the visitor's IP address (sign-in and sign-up) or user id (password change), a count and a reset time	Brute-force protection

Table	Personal fields	Purpose
`audit_log`	user id and email, action, item, details, time	Who changed what
`collection_items`	the id of the user who added or removed it; purchase price and date, notes, grading cert number	The collection
`sales`, `binders`, `wants`	the id of the user who created it	Sales log, binders, want list

Personal or sensitive data and how it is protected.

- **Passwords:** never sent or stored in plain text (above).
- **Session tokens:** random, and stored only as hashes.
- **Brute-force limits:**
  - Sign-in is rate-limited to 30 attempts per IP address per 15 minutes.
  - Sign-up is rate-limited to 5 per IP address per hour.
  - An account locks for 15 minutes after 8 wrong passwords.
- **Roles:** every API request is checked against the user's role (Pending, Viewer, Editor, Admin), so hiding a button in the app is never the only protection.
- **Who can call the API:** only the configured web addresses (`ALLOWED_ORIGINS`), over HTTPS.
- **Deletion:** removing a card moves it to a trash that only an admin can empty. Deleting a user removes their sessions, and their items stay with no owner recorded.
- **Backups:** admins can download all data as JSON, or the collection as CSV. D1 Time Travel can restore the database to any point in the last 7 days.

Known gaps (from reading the code; not yet fixed):

- Expired sessions and rate-limit rows are never deleted. Rate-limit keys contain IP addresses, so those stay in the database indefinitely.
- The audit log keeps a user's email address after their account is deleted.
- The live instance allows anyone to create a Pending account, which stores their email address. It has no privacy notice.
- Cloudflare Workers observability (request logging) is enabled in `api/wrangler.jsonc`. What it records and for how long is set by Cloudflare. **TODO (Ryley): check.**
- Where Cloudflare stores the D1 database: **TODO (Ryley): check in the Cloudflare dashboard.**

Intended uses. Managing one shared collection for a small group of trusted users.

Out-of-scope uses. Any use of users' personal data other than running the app, such as marketing, sharing or profiling.

Known limitations. One shared collection per deployment, with no per-user collections. No retention policy is enforced (see the gaps above).

Maintenance.

- The schema changes only through numbered migrations, which `npm run deploy` applies.
- **TODO (Ryley): decide on a retention period.** A scheduled clean-up of expired sessions and rate-limit rows would close the first gap above.
- Owner: Ryley Griffiths.

5. Simulated page-photo evaluation set

Summary. Synthetic photos of binder pages, with a record of which card is in each pocket. They are used to measure the page scanner. They are generated on demand, not committed: the evaluation in this repository used 140 pages (seeds 1, 2 and 3).

Source and provenance. `eval/page-scan/generate.py` composes each page from the catalogue's card pictures, which it downloads from TCGdex into a local cache. It then adds pocket plastic haze, seams, sleeves, lamp glare, misalignment, tilt, noise, blur and JPEG compression.

Licence and terms. Derived from the card pictures (see dataset 1), so the generated images are not redistributed.

Collection and processing.

- **Pages:** each page holds consecutive cards from one set.
- **Pockets:** about 10% are left empty and about 10% hold a random card from another set; about 30% of cards are in penny sleeves.
- **Repeatable:** the output is the same for a given seed and catalogue build.

Schema. `truth.json` holds:

- the seed
- the catalogue build used (time and hash)
- one entry per page: its file name, the pocket layout (columns and rows), the card that belongs in each pocket, the card actually in it (or null if empty), and whether each card is sleeved

Personal or sensitive data. None.

Intended uses. Measuring and comparing versions of the page scanner.

Out-of-scope uses. Claims about accuracy on real photographs, and training models: the photos come from the same pictures as the fingerprints.

Known limitations and biases.

- Simulated, not photographed. Real cards differ from their scans (colour, print variation, wear, holographic foil), and real binder plastic and phone cameras behave differently. Results are therefore expected to be optimistic.
- Wrong cards come from other sets, which is easier than a reprint of the right card.
- Results depend on the catalogue build (recorded in `truth.json`).

Maintenance. The generator and scorer live in `eval/page-scan/`. Re-run them after changing the scanner. Owner: Ryley Griffiths.